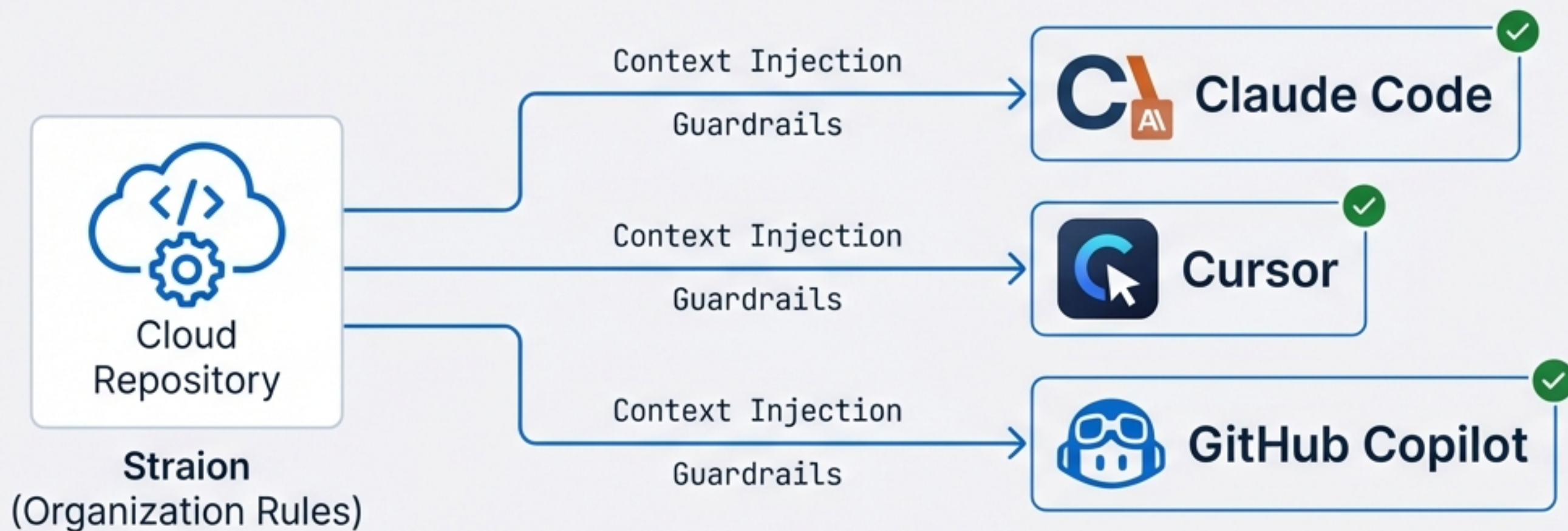


Straion: AIコーディングエージェントのための「ルール」管理プラットフォーム

組織のエンジニアリング・スタンダードを、AIの「脳」に動的にインストールする



エグゼクティブ・サマリー

What (概要)

AIエージェント (**Claude, Cursor, Copilot**) のルールを一元管理するSaaS。散在する **.md** ファイルをクラウドに集約し、組織全体のナレッジとして運用。

How (仕組み)

Semantic Matching:
タスクの内容に応じて、必要なルールだけをAIに動的に注入 (**Dynamic Injection**)。

Plan Validation:
コード生成前に、AIの「計画」段階でルール違反を**自動検知・ブロック**。

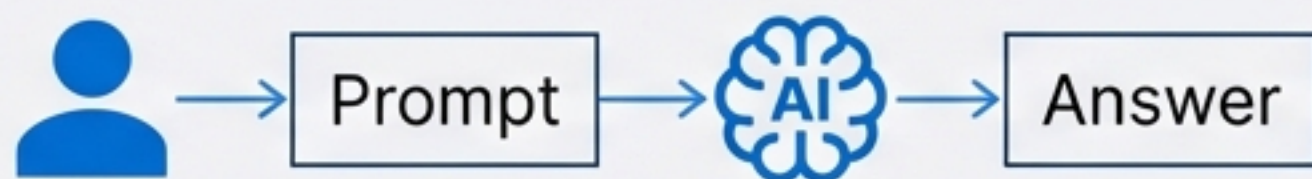
Why (効果)

レビュー工数の削減:
AIが最初から「正解」に近いコードを書くため、手戻りが激減。

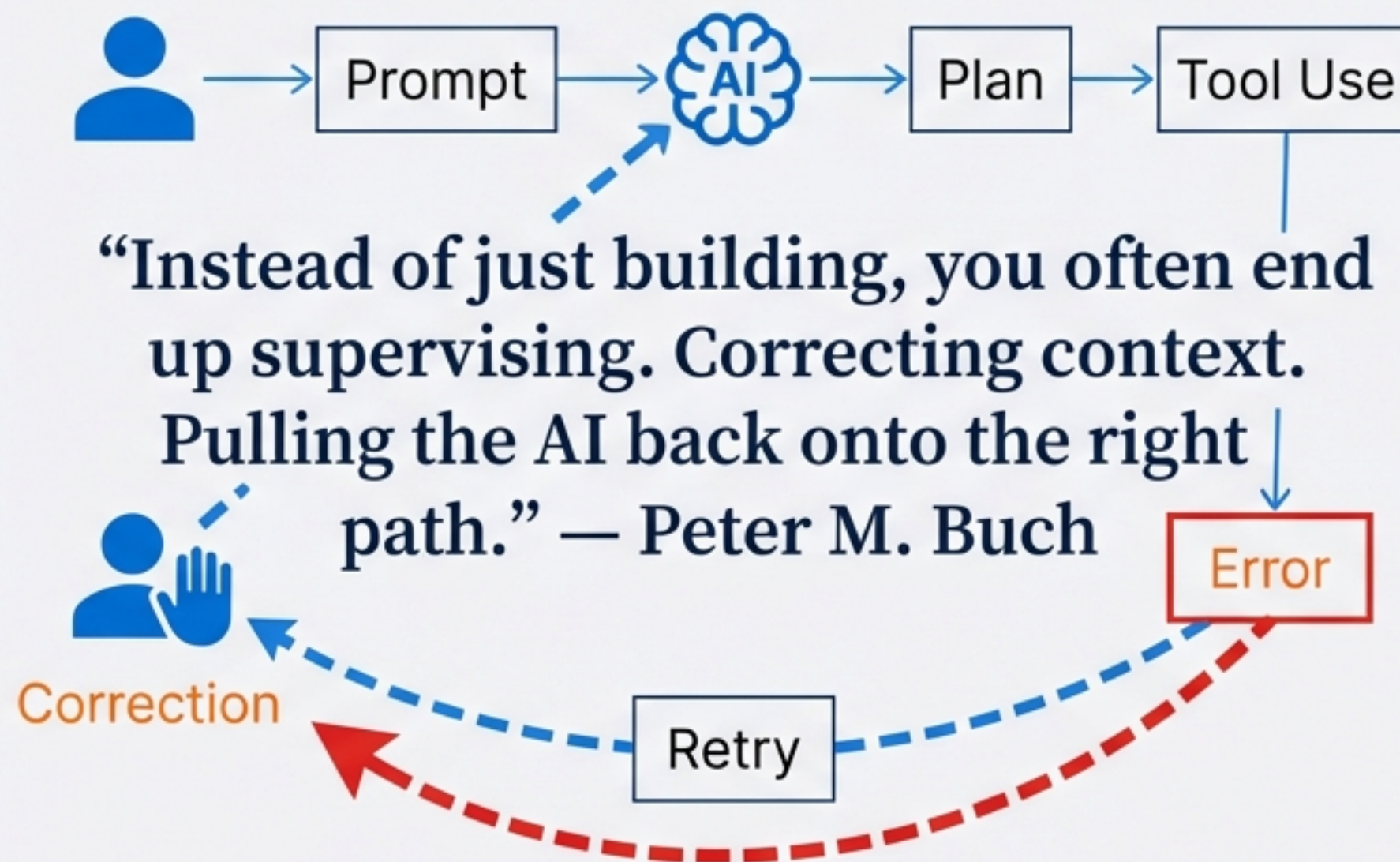
セキュリティと品質の担保:
経験の浅いメンバーやAIでも、組織のセキュリティ基準 (PII扱いなど) を**遵守**。

「チャット」から「エージェント」へ：新たなボトルネックの発生

Chat Era (Legacy)

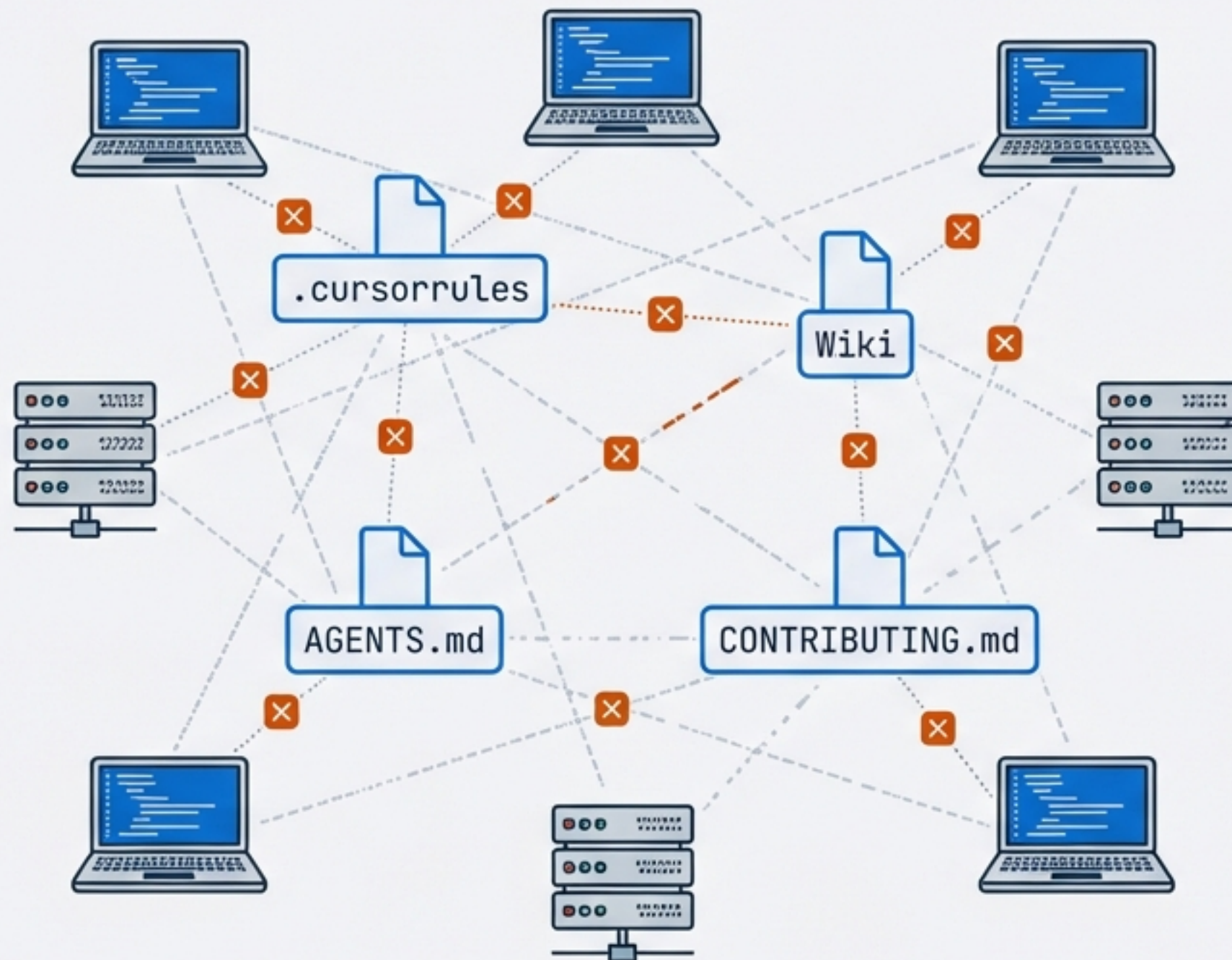


Agent Era (Now)



AIエージェントの能力は向上したが、エンジニアは「監視役（Babysitting）」に追われている。

静的ファイル管理 (Static Files) の限界



1. 断片化 (Fragmentation)

ルールがWikiや個人の頭の中、あるいは複数のリポジトリに散らばり、更新が同期されない。

2. トークンの無駄 (Token Waste)

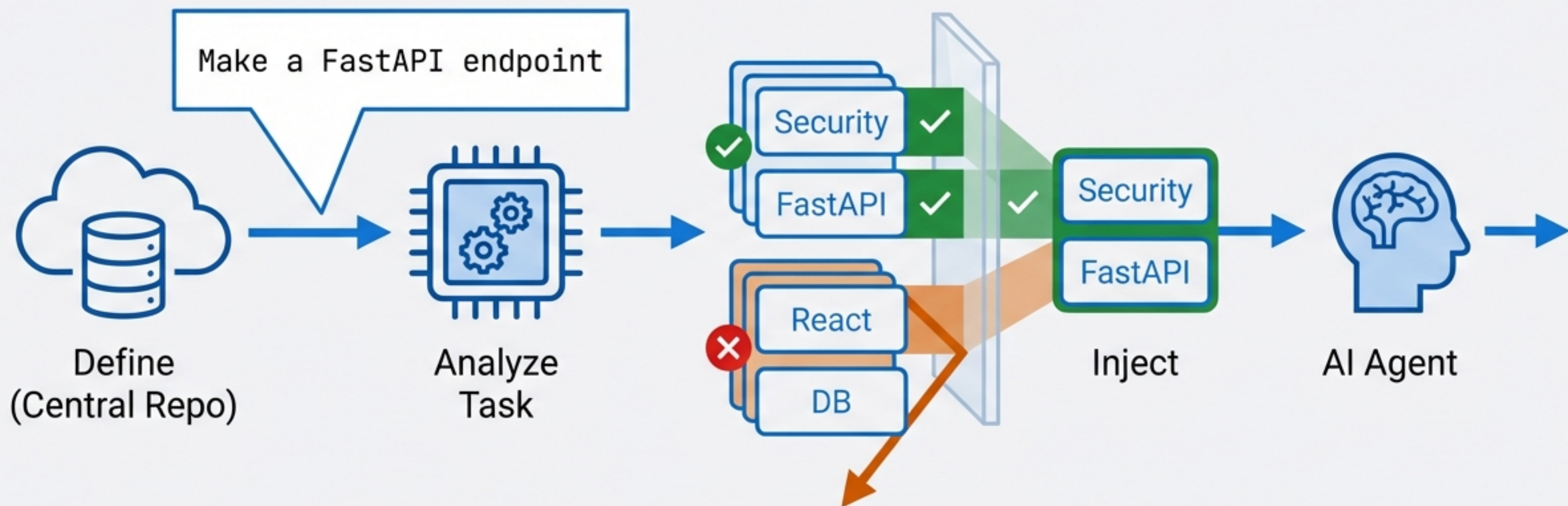
すべてのルールをコンテキストウィンドウに流し込むと、AIが混乱し、コストも高む。

3. 静的 (Static)

AIは「今のタスク」にどのルールが必要か判断できず、無関係なルールまで参照してしまう。

Straion: 必要なルールを、必要な時だけ

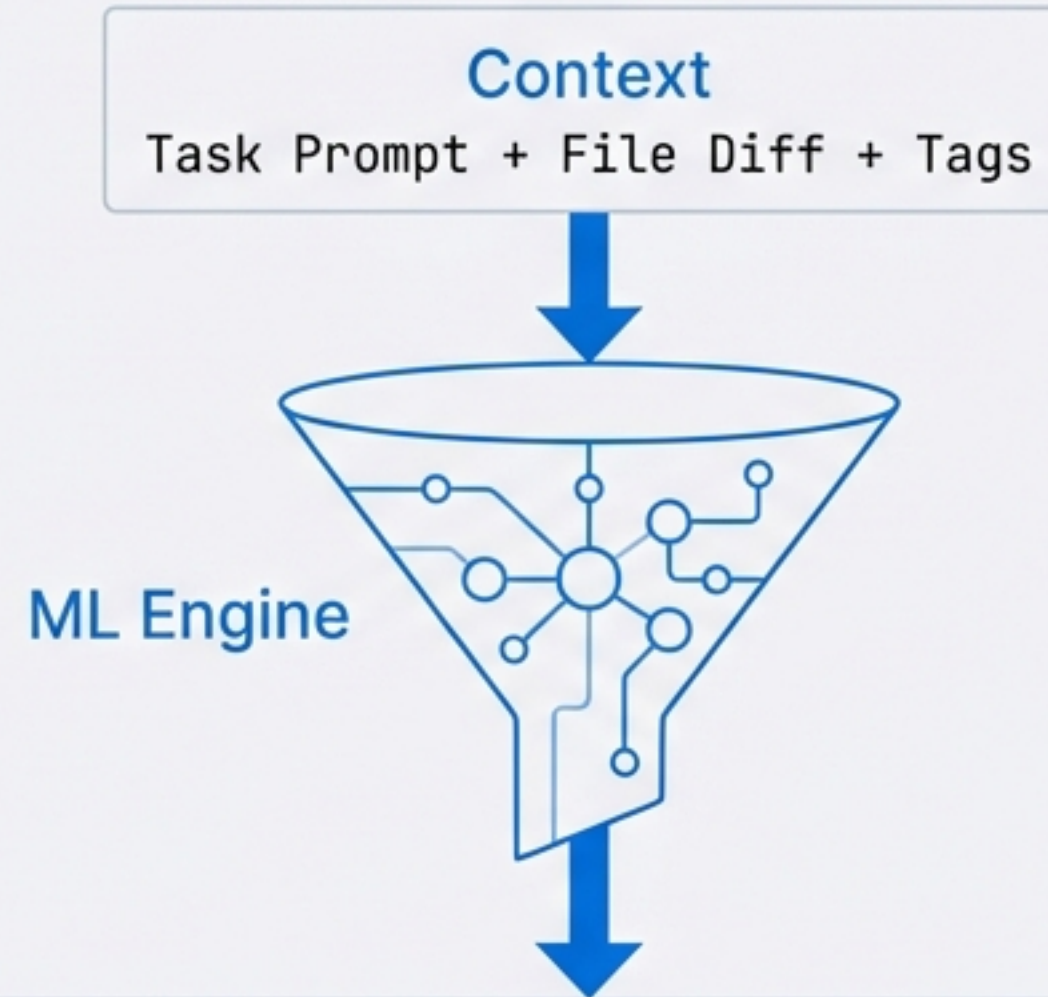
Dynamic Context Injection (動的コンテキスト注入)



タスクを解析し、関連するルールだけを抽出して注入。常に最適化された「脳」でタスクを実行。

コア技術：Semantic Matching

Filter Pipeline



単なるキーワード検索ではなく、タスクの「意味」を理解してルールを適用。AIのハルシネーション（幻覚）とコンテキスト汚染を防ぐ。

1	Rule: API Security	94% ✓	Score: 0.94 (INJECT)
2	Rule: Python Style	88% ✓	Score: 0.88 (INJECT)
3	Rule: Frontend UI	20% ✕	Score: 0.20 (IGNORE)

「計画 (Plan)」段階での事前ブロック

```
> AI Plan: I will log the user object including email for debugging.  
> ⚠️ Straion Alert: Violation Detected (Rule: No PII in logs)  
> AI Correction: Revised plan. I will hash the user ID before logging.  
> ✅ Straion: Plan Approved.  
> JetBrains Mono font  
> Generating Code: logger.info(f'User {user_id_hash} logged in')
```

コードが書かれる前に、AIの思考プロセス (Plan) を検証。技術的負債の発生源を断つ。

どのようなルールを強制できるか？



Security & Compliance

APIキーのハードコード禁止、PII（個人情報）のログ出力のログ出力禁止。

 ~~const apiKey = 'sk-123...'~~



const apiKey = process.env.API_KEY



Architecture

Next.js App Routerの強制、階層化アーキテクチャの遵守。



Coding Style

命名規則、特定のライブラリ使用の強制
(例: 'Use shadcn/ui components only').

比較：Straion vs. Ruler (OSS) vs. Static Files

Solution	Features	Maintenance / Validation	Target
Static Files (.md)	No Centralization No Dynamic Injection	Maintenance: High	None
Ruler (OSS)	Local/Git Centralization Static Injection (All Rules)	Plan Validation: No	Solo Devs / Offline
Straion (SaaS)	Cloud Centralization Dynamic Injection (ML)	Plan Validation: Yes  Enterprise Logs	Teams / Enterprises

Rulerはファイルを「配る」ツール。Straionはルールを「知的に適用する」プラットフォーム。

推奨される導入戦略

The Hybrid Strategy

Intelligence Layer: Straion
Dynamic Injection & Compliance

Base Layer: Ruler (OSS)
Static Sync via Git

👤 Solo / Hobby

Ruler + Static Filesで十分。

Team / Organization

Review Overhead（レビューの負荷）
削減のため、Straionが必須。

Enterprise

監査ログとセキュリティ強制のため、
Straion一択。

市場の評価と実績



@ai_negi_lab_com

SDKでルールを注入した結果、規約を無視した『手抜きコード』が一瞬で『チーム標準』に化けた。技術的負債が劇的に減る。

(AI Tech Lead / Blogger)

概念だけでなく、実務（FastAPI開発など）で効果が実証されている。

導入は5分で完了

Terminal

```
$ npm install -g @straion/cli
...
$ straion import ./existing-rules.md
...
✓ Successfully loaded 12 requirement sets.

$ /add-skill straion:validating-rules
→ Dynamically injected: security + fastapi rules
```

1 CLIをインストール

2 既存ルールを
インポート

3 エージェントに
Skillを追加

AI時代の「組織の記憶」を構築する



コーディング規約は、もはやWikiに書くものではなく、
AIのコンテキストに組み込む「ソフトウェア依存関係」である。

Start with Free Tier: Import existing .md files & Test Plan Validation

Don't let AI dilute your engineering culture; encode it.